

Checklist de calidad : Tipos de datos básicos, métodos, atributos y variables

A. Rutinas o métodos

ID	Criterio	Descripción	Justificación	Problema que ayuda a evitar
A-01	Razón de existencia	Cada método tiene un porqué es implementado.	Todo método debe responder a una necesidad del negocio o de su clase.	Métodos innecesarios que complican el diseño sin aportar nada.
A-02	Propósito bien definido	Cada método tiene un propósito bien definido.	Un método que hace una sola cosa es más fácil de entender, probar y reutilizar.	Métodos que mezclan tareas y son difíciles de mantener.
A-03	Nombre bien definido	El nombre del método dice exactamente lo que hace.	El nombre es la primera documentación de un método.	Tener que leer la implementación para saber qué hace.
A-04	Nombres con verbos	Los nombres de los métodos usan verbos.	Un método representa una acción, y el verbo lo hace evidente.	Nombres que parecen datos y no operaciones.

A-05	Poca dependencia	El método depende poco del resto del programa.	Cuanto menos dependa de otras partes, más fácil es probarlo y modificarlo.	Que un cambio en otra parte del programa rompa el método.
A-06	Parámetros con sentido	Los parámetros dentro del método tienen sentido en conjunto.	Parámetros relacionados forman una interfaz coherente.	Métodos que reciben datos sin relación entre sí.
A-07	Orden lógico y constante	Los parámetros siguen un orden lógico y siempre el mismo.	Un orden uniforme hace predecible el uso de los métodos.	Errores por intercambiar argumentos al llamar métodos parecidos.
A-08	Pocos parámetros	El método tiene pocos parámetros, de preferencia no más de 7.	Muchos parámetros suelen indicar que el método hace demasiado.	Métodos difíciles de llamar y de entender.
A-09	Partes separadas	Las partes que se pueden separar en su propio método están separadas.	Separar facilita reutilizar y probar cada parte.	Métodos largos con lógica repetida o mezclada.
A-10	Longitud según lo que hace	La longitud del método depende de lo que hace.	La longitud debe ser proporcional a la tarea que realiza.	Métodos excesivamente largos o fragmentados sin necesidad.
A-11	Parámetros usados	Se usan todos los parámetros del método.	Un parámetro sin usar señala un error de diseño o código sobrante.	Interfaces confusas con datos que no sirven.
A-12	Valor devuelto válido	Si es una función, devuelve un valor válido en todos los casos.	Quien llama al método debe poder confiar en su resultado.	Resultados nulos o erróneos en algún caso.
A-13	Convención para operaciones parecidas	Las operaciones parecidas se nombran con la misma convención.	Una convención común hace reconocible el tipo de operación.	Nombres inconsistentes para operaciones equivalentes.

B. Variables

ID	Criterio	Descripción	Justificación	Problema que ayuda a evitar
B-01	Parámetros validados	Los parámetros de entrada de los métodos se validan cuando es necesario.	Los datos que llegan del exterior no siempre son correctos.	Datos inválidos que dejan el sistema en un estado incorrecto.
B-02	Declaración cerca del uso	Las variables se declaran cerca de donde se usan por primera vez.	Facilita seguir el código y entender para qué sirve cada variable.	Variables declaradas lejos de su uso, difíciles de rastrear.
B-03	Inicialización antes de usarse	Las variables se inicializan antes de usarse.	Garantiza que siempre tengan un valor conocido.	Errores por usar variables sin valor.
B-04	Contadores y acumuladores	Los contadores y acumuladores se inicializan y reinician bien.	Un acumulador mal iniciado arrastra valores anteriores.	Totales o conteos incorrectos.
B-05	Alcance mínimo	Cada variable tiene el menor alcance posible.	Cuanto más acotada, menos partes del código pueden afectarla.	Modificaciones accidentales desde otras partes del programa.
B-06	Usos cercanos	Los usos de una variable están cerca unos de otros.	Agrupar los usos facilita entender su comportamiento.	Código difícil de seguir por tener que saltar entre partes lejanas.
B-07	Vida corta	Las variables no viven más tiempo del necesario.	Reduce el riesgo de usarlas con valores antiguos.	Valores desactualizados y consumo innecesario de memoria.
B-08	Sin variables globales	Se evitan las variables globales innecesarias.	Los datos globales los puede modificar cualquier parte del programa.	Dependencias ocultas y errores difíciles de rastrear.

B-09	Sin variables sin usar	No hay variables sin usar.	Una variable sin uso indica código sobrante o un error.	Código confuso y elementos que no aportan.
B-10	Propósito claro	Cada variable tiene un propósito claro.	Facilita entender el código sin explicaciones adicionales.	Variables cuya función no se entiende.
B-11	Sin reutilización	Una variable no se reutiliza para cosas distintas.	Un solo significado por variable evita confusiones.	Errores por interpretar un valor con un significado equivocado.
B-12	Valores que cambian en configuración	Los valores que pueden cambiar se leen de una configuración y no se escriben en el código.	Permite cambiar valores sin modificar el programa.	Tener que editar el código cada vez que cambia un valor.

C.Nombres de variables

ID	Criterio	Descripción	Justificación	Problema que ayuda a evitar
C-01	Nombres adecuados	Los nombres dicen bien lo que representan.	El nombre es la forma más directa de documentar una variable.	Confundir qué guarda cada variable.
C-02	Conceptos del problema	Los nombres usan conceptos del problema, no detalles técnicos.	Conecta el código con el lenguaje del negocio.	Nombres técnicos que no dicen qué significa el dato para el negocio.
C-03	Longitud adecuada	La longitud de los nombres es adecuada.	Ni muy cortos para entenderse ni tan largos que dificulten la lectura.	Nombres crípticos o excesivamente largos.
C-04	Sin nombres vagos	Se evitan los nombres vagos o poco entendibles.	Nombres como dato o temp no explican nada.	Nombres que obligan a adivinar el propósito.

C-05	Índices descriptivos	Los índices de ciclos anidados o complejos tienen nombres descriptivos.	En ciclos anidados las letras sueltas se confunden.	Errores al intercambiar índices.
C-06	Variables temporales con sentido	Las variables temporales tienen nombres con sentido cuando su propósito no es obvio.	Una variable temporal sigue necesitando ser entendida.	Código que oculta pasos intermedios importantes.
C-07	Booleanas legibles	Las variables booleanas se leen bien cuando valen verdadero.	Un nombre en positivo se interpreta sin esfuerzo.	Dobles negaciones que confunden la condición.
C-08	Enumeraciones claras	Los nombres de las enumeraciones dejan claro cada estado.	Cada valor debe indicar sin ambigüedad qué estado representa.	Estados confusos o que se interpretan de varias formas.
C-09	Constantes por significado	Las constantes se nombran por lo que significan, no por su valor.	El valor puede cambiar, el significado no.	Nombres que quedan incorrectos cuando cambia el valor.
C-10	Elementos parecidos	Los elementos parecidos se nombran de forma parecida.	Una misma cosa con un mismo nombre en todo el proyecto.	Que un mismo concepto tenga varios nombres.
C-11	Misma convención	Todo el proyecto usa la misma convención de nombres.	Una convención única hace el código uniforme.	Mezcla de estilos que dificulta la lectura.
C-12	Sin abreviaturas innecesarias	Se evitan las abreviaturas innecesarias y los nombres que se confunden fácil.	Las abreviaturas no siempre se entienden igual.	Nombres que se interpretan de forma distinta según quien los lea.
C-13	Abreviaturas documentadas	Las abreviaturas que se usan están documentadas.	Si se usan, deben tener un significado conocido por todos.	Abreviaturas sin explicación.

C-14	Calificadores al final	En los nombres de valores calculados, el calificador va al final.	Un orden fijo hace los nombres predecibles.	Nombres con el calificador en posiciones distintas.

D.Tipos de datos básicos

ID	Criterio	Descripción	Justificación	Problema que ayuda a evitar
D-01	Sin números mágicos	No hay números mágicos en el código.	Un número suelto no dice qué significa.	Valores que nadie sabe de dónde salen ni cómo cambiar.
D-02	Constantes con nombre	Los valores constantes importantes tienen nombre.	Un nombre da significado y centraliza el valor.	Cambiar un valor en muchos lugares.
D-03	Compilación sin advertencias	El código compila sin advertencias.	Las advertencias suelen señalar posibles errores.	Errores que pasan inadvertidos.
D-04	Desbordamiento de enteros	Se considera el desbordamiento de enteros.	Un entero tiene un límite máximo.	Resultados incorrectos cuando un valor excede el límite.
D-05	Precisión de decimales	Se tienen en cuenta la precisión y el redondeo de los números decimales (float y double).	Los decimales binarios no siempre representan valores exactos.	Diferencias de redondeo en los cálculos.
D-06	Booleanos para sí o no	Se usan booleanos para condiciones de sí o no.	El tipo boolean expresa exactamente dos estados.	Estados con más de dos valores guardados como booleano.
D-07	Enumeraciones para valores fijos	Se usan enumeraciones cuando hay un conjunto fijo y conocido de valores.	Restringen los valores posibles a los válidos.	Valores inválidos o mal escritos, como textos libres.

D-08	Dinero sin errores de precisión	Los cálculos con dinero evitan los problemas de precisión de los decimales.	Los importes deben ser exactos.	Centavos perdidos o sobrantes en totales.
D-09	Valores inválidos en enumeraciones	Las pruebas con enumeraciones verifican valores inválidos.	Comprueba que el sistema rechace estados no permitidos.	Que un valor inválido se acepte sin detectarse.
D-11	Un tipo por cada dato que cambia	Se usa un tipo distinto por cada clase de dato que puede cambiar.	Un tipo propio concentra las reglas de ese dato.	Reglas de un mismo dato repetidas en muchas clases.
D-12	Tipos del mundo real	Los nombres de los tipos se orientan a las entidades reales que representan.	El código refleja el lenguaje del negocio.	Tipos abstractos que no corresponden a nada real.